Introduction

The purpose of this document is to evaluate potential security weaknesses and implement security controls within the mobile app's infrastructure.

Scope of Work

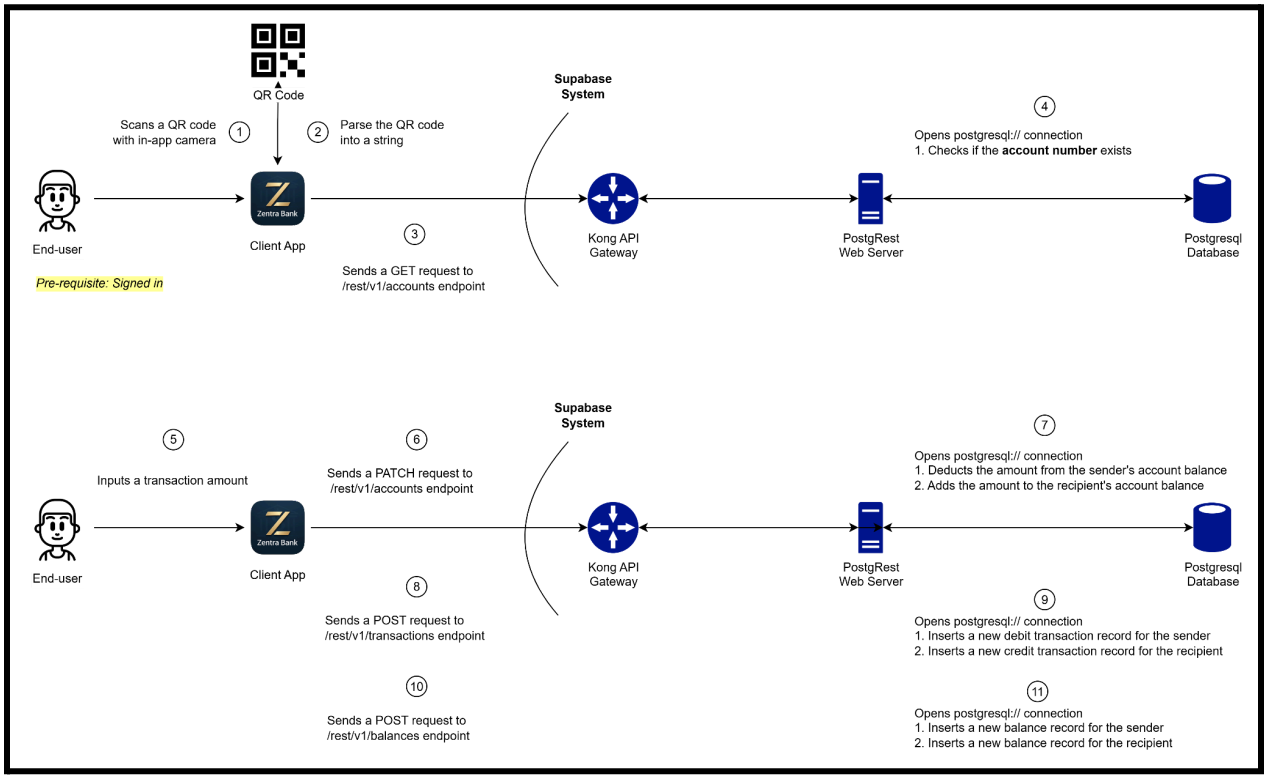
This exercise focuses on 1 user story within the Zentra Bank mobile app:

1. As a user, I want to **transfer funds using "PayNow" thru QR code** so that I can have another alternative to send money to friends, family, or service providers

Out of Scope

This exercise strictly focuses on the mobile application and its functionalities. Any components outside the mobile app, which includes but not limited to REST APIs, server infrastructure, remote database, and external services, are not within the scope.

User story 5



The data flow for this user story is as follows:

"As a user, I want to transfer funds using "PayNow" thru QR code so that I can have another alternative to send money to friends, family, or service providers."

1. The End-user scans a QR code with the in-app camera
2. The Client App (CA) parses the QR code into an account number string and sends a check request via REST API to the Web Server (WS) to verify whether it exists
3. The Web Server (WS) checks 1) if the account number exists against the `accounts` table in the Database (DB) via Postgres connection

```
GET /rest/v1/accounts?select=*&account_number=eq.00177714179 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
accept: application/json
accept-profile: public
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc
2F1dGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhd
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZ
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92Z
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0Z
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyL
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
x-client-info: supabase-js-react-native/2.49.5-next.1
```

4. The End-user inputs a transaction amount

5. To complete the transfer, the Client App (CA) first sends a request to update accounts' balances via REST API to the Web Server (WS) to 1) deduct the amount from the sender's account balance and 2) add the amount to the recipient's account balance

6. The Web Server (WS) makes an update query to the **`accounts`** table in the Database (DB) via Postgres connection. The Database (DB) then 1) deducts the amount from the sender's account balance and 2) adds the amount to the recipient's account balance

```
PATCH /rest/v1/accounts?account_id=eq.4fc4ca5e-09d2-433c-9321-add4c91aa6a2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cH
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRw
2F1dGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRo
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJw
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwi
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyLTljNTctMzc1
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 21

{"current_balance":2}
```

```
PATCH /rest/v1/accounts?account_id=eq.15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHN
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRw
2FldGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRo
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJw
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwi
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyLTljNTctMzc1
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 23

{"current_balance":18}
```


7. The Client App (CA) will also send a request to insert new transaction records via REST API to the Web Server (WS) for audit trail

8. The Web Server (WS) makes an insert query to the **`transactions`** table in the Database (DB) via Postgres connection. The Database (DB) then inserts 1) a debit transaction record for the sender and 2) a credit transaction record for the recipient


```
POST /rest/v1/transactions?columns="account_id","booking_datetime","amount","currency","credit_debit","description","transaction_type","status" HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6MjA2NDA1N
BmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJzdWIiOiI
C00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJleHAiOjE3NTY4OTU4MzIsImlhdCI6MTc1Njg5MjIzMiwidXNlcl9tZXRhZGF0YSI6eyJlbWFpbCI6InRlc3RuaWNAbG1haWwuY29tIn1
tcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 430

[{
  "account_id": "4fc4ca5e-09d2-433c-9321-add4c91aa6a2",
  "booking_datetime": "2025-09-03T09:49:48.381Z",
  "amount": 1,
  "currency": "SGD",
  "credit_debit": "Debit",
  "description": "Transfer to 00177714179",
  "transaction_type": "Online Transfer",
  "status": "Booked"
},{
  "account_id": "15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2",
  "booking_datetime": "2025-09-03T09:49:48.381Z",
  "amount": 1,
  "currency": "SGD",
  "credit_debit": "Credit",
  "description": "Transfer from 00139540222",
  "transaction_type": "Online Transfer",
  "status": "Booked"
}]
```


9. The Client App (CA) will also send a request to insert new balance records via REST API to the Web Server (WS) for audit trail

10. The Web Server (WS) makes an insert query to the **`balances`** table in the Database (DB) via Postgres connection. The Database (DB) then inserts a new balance record for 1) the sender and 2) the recipient

```
POST /rest/v1/balances?columns="account_id","balance_type","amount","currency","credit_debit","reference_date" HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpY
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN
zdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImFldGhlbnRpY2F0ZWQiLCJleHAiOjE3NTY4OTU4MzIsImlhdCI6M
GF0YSI6eyJlbWFpbCI6InRlc3RuaWNAbGlhaWwuY29tIn19.XmKw04_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 373

[{
  "account_id": "4fc4ca5e-09d2-433c-9321-add4c91aa6a2",
  "balance_type": "InterimAvailable",
  "amount": 2,
  "currency": "SGD",
  "credit_debit": "Debit",
  "reference_date": "2025-09-03T09:49:48.514Z"
},{
  "account_id": "15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2",
  "balance_type": "InterimAvailable",
  "amount": 18,
  "currency": "SGD",
  "credit_debit": "Credit",
  "reference_date": "2025-09-03T09:49:48.514Z"
}]
```

Bonus user story

Insert diagram here

The data flow for this user story is as follows:

“As a user, I want to generate my own QR code so others can have an alternative means to send me money easily.”

1. The End-user generates a QR code by clicking a button
2. The Client App (CA) sends a request via REST API to the Web Server (WS) to retrieve the user’s account number
3. The Web Server (WS) queries the `account_parties` table in the Database (DB) via Postgres connection to confirm that the account exists for the customer.

```
GET
/rest/v1/account_parties?select=account_id&customer_id=eq.b15e81f3-7a2f-4f9
9-bef9-cdeb21464417 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
Accept: application/json
Accept-Profile: public
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW
9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6M
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJp
c3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJ
zdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRoZW
50aWNhdGVkIiwiZXhwIjoxNzU3NDY5MzE0LCJpYXQiOjE3NTc0NjU3MTQsImVtYWlsIjoidGVzd
G5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJwcm92aWRlciI6ImVt
YWlsIiwicHJvdmlkZXJzIjpbImVtYWlsIl19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGV
zdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6dHJ1ZSwicGhvbmVfdmVyaWZpZWQiOm
ZhbHNlLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZ
SI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwiOiJhYWwxIiwiYW1yIjpbeyJtZXRob2QiOiJwYXNzd29y
ZCIsInRpbWVzdGFtcCI6MTc1NzQ2NTcxNH1dLCJzZXNzaW9uX2lkIjoiYmVhZGNlZjItNjdmMy0
0OWIzLWExZGMtZWJiNjkzYWMyNjBlIiwiaXNfYW5vbnltb3VzIjpmYWxzZX0.uaIL3wqxHooPIP
aLJ_cZx6CGQ8XSCGqn1F3XSmjwvm0
x-client-info: supabase-js-react-native/2.49.5-next.1
```

4. The Client App (CA) then sends another request via REST API to the Web Server (WS) to retrieve account number from the `accounts` table in the Database (DB) via Postgres connection.

```
GET
/rest/v1/accounts?select=*&account_id=eq.4fc4ca5e-09d2-433c-9321-add4c91aa6
a2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
Accept: application/vnd.pgrst.object+json
Accept-Profile: public
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW
9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6M
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJp
c3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJ
zdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRoZW
50aWNhdGVkIiwiZXhwIjoxNzU3NDY5MzE0LCJpYXQiOjE3NTc0NjU3MTQsImVtYWlsIjoidGVzd
G5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJwcm92aWRlciI6ImVt
YWlsIiwicHJvdmlkZXJzIjpbImVtYWlsIl19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGV
zdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6dHJ1ZSwicGhvbmVfdmVyaWZpZWQiOm
ZhbHNlLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZ
SI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwiOiJhYWwxIiwiYW1yIjpbeyJtZXRob2QiOiJwYXNzd29y
ZCIsInRpbWVzdGFtcCI6MTc1NzQ2NTcxNH1dLCJzZXNzaW9uX2lkIjoiYmVhZGNlZjItNjdmMy0
0OWIzLWExZGMtZWJiNjkzYWMyNjBlIiwiaXNfYW5vbnltb3VzIjpmYWxzZX0.uaIL3wqxHooPIP
aLJ_cZx6CGQ8XSCGqn1F3XSmjwvm0
x-client-info: supabase-js-react-native/2.49.5-next.1
```

5. The Client App (CA) encodes the account number as a string into a QR code, and displays it on screen for others to scan

Note: the logical data flow should be user logged in then generate a QR code. The flow is as such to illustrate the information needed to generate the QR code.

Methodology

The methodology for this exercise follows a structured approach:

1. Create data flow diagrams to map how data moves within the mobile app
2. Use security testing tools and threat modeling to identify vulnerabilities
    a. Mobile Application Security Framework (MobSF)

3. Evaluate the impact of the identified vulnerability, how they could affect the app
4. Research remediation measures and show their effectiveness

Findings Overview

Scope of Work..... 1
Out of Scope..... 1
User story 5..... 2
Bonus user story..... 6
Methodology..... 8
Template..... 10

Template

Legend
- White background cells: For interns to fill
- Blue / Orange / Green background cells: For Supervising officer to fill

Issue: App can be installed on a vulnerable unpatched Android version
Issue ID: US01-MOBSF01. My app allows installation by devices running on Android 7.0 and above. Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit. **Cause**: My app allows installation by devices running on Android 7.0 and above. **Effect**: Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit.
Demonstration: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App?
Control ID: US01-MOBSF01-C01. Remove threat by ensuring the app installs only on devices running Android versions with active security support. **Control**: Remove threat by ensuring the app installs only on devices running Android versions with active security support.
Implemented at: Client App / Authentication Server / Database **Demonstration**: - Show how the control mitigates the threat. - Explain with evidence on how you implement the control?
Status: tried (implemented) / tested (tested for effectiveness)
Another implementation

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

Control ID:
US01-MOBSF01-C02. Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Control:
Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

No new risks that arise as a result of implementing user stories

Issue: App creates a temp file. Sensitive information should never be written to a temp file.

Detailed Description:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific file that was flagged out by MobSF:
`com/mrousavy/camera/core/utils/OutputFile.java, line(s) 79`

This is not an issue to the App, because this file within the react-native-vision-camera package will never be executed in our workflow. The specific file used in this package is useCodeScanner.ts, to scan QR codes. No temporary files were created. See below for implementation.

app/scanScreen.jsx +137

```
@@ -0,0 +1,137 @@
+ import { Camera, useCameraDevice, useCodeScanner, useCameraPermission } from "react-native-vision-camera";
+ import { useEffect, useState, useRef } from "react";
+ import { View, Text, TouchableOpacity, StyleSheet } from "react-native";
+ import { ScanOverlay} from "../app/component/ScanOverlay";
+ import { useRouter, useLocalSearchParams } from "expo-router";
+
+ export default function ScanScreen() {
+
+     const router = useRouter();
+     const { userAccountNumber, userAccountBalance } = useLocalSearchParams();
+     const device = useCameraDevice('back');
+     const { hasPermission, requestPermission } = useCameraPermission();
+     const codeScanner = useCodeScanner({
+         codeTypes: ['qr'],
+         onCodeScanned: (code) => {
```

```
import { useCallback, useMemo, useRef } from 'react'
import type { Code, CodeScanner, CodeScannerFrame } from '../types/CodeScanner'

export function useCodeScanner(codeScanner: CodeScanner): CodeScanner {
  const { onCodeScanned, ...codeScannerOptions } = codeScanner

  // Memoize the  function once and use a ref on any identity changes
  const ref = useRef(onCodeScanned)
  ref.current = onCodeScanned
  const callback = useCallback((codes: Code[], frame: CodeScannerFrame) => {
    ref.current(codes, frame)
  }, [])

  // CodeScanner needs to be memoized so it doesn't trigger a Camera Session re-build
  return useMemo(
    () => ({
      ...codeScannerOptions,
      onCodeScanned: callback,
    }),
    // eslint-disable-next-line react-hooks/exhaustive-deps
    [JSON.stringify(codeScannerOptions), callback],
  )
```

More information
This issue could potentially result in CWE-377 (Insecure Temporary File) and CWE-379 (Creation in Directory with Insecure Permission). Temp files could be created in external storage rather than internal storage.

Android 10 and below, allows other applications with READ_EXTERNAL_STORAGE / WRITE_EXTERNAL_STORAGE to read media created / stored in external storages. If these temp files are saved there, it may result in the leak of those files.

Added a logging function to find out if the OutputFile function is executed during the user story workflow. No log was produced, concluding that this function did not execute during the user story workflow.

```
data class OutputFile(val context: Context, val directory: File, val extension: String) {
  val file = File.createTempFile("mrousavy", extension, directory)

  init {
    android.util.Log.d("RNVC/OutputFile", "dir=${directory.absolutePath}, ext=$extension, cacheDir=${context.cacheDi
    if (directory.absolutePath.contains(context.cacheDir.absolutePath)) {
      // If this is a temp file (inside temp directory), the file will be deleted once the app closes
      file.deleteOnExit()
    }
  }
}
```

```
PROBLEMS 65   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

nictjh@r-50-101-25-172 android-mobile-banking-app % adb logcat | grep "RNVC/OutputFile"
* daemon not running; starting now at tcp:5037
* daemon started successfully
```

Although the app imports only specific modules from the “react-native-vision-camera” dependency, the APK bundle may still include all modules and files from that dependency.

This is why files that were never explicitly imported can still be found within the APK. See below screenshot.

Furthermore, upon further investigation, the output file in question has dependencies on many other files including the Camera module within the dependency, making it impossible to take it out without removing the core functionality of QR code scanning. See below screenshot for the files dependent on the output file.

Note: use grep -R "Lcom/mrousavy/camera/core/utils/OutputFile;" -n out_apktool

- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/TakePhotoOptions.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/RecordVideoOptions$Companion.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/TakePhotoOptions$Companion.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/TakeSnapshotOptions$Companion.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/RecordVideoOptions.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/types/TakeSnapshotOptions.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/CameraSession_VideoKt.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/core/utils/OutputFile.smali` ← class definition
- `out_apktool/smali_classes2/com/mrousavy/camera/core/CameraSession_PhotoKt.smali`
- `out_apktool/smali_classes2/com/mrousavy/camera/react/CameraView_TakeSnapshotKt.smali`

For remediation, it’s recommended that system owners either accept this risk, consider switching to an alternative library with a smaller footprint, or build a custom solution tailored to their needs.

Issue: android.permission.CAMERA (dangerous): take pictures and videos - Allows application to take pictures and videos with the camera. This allows the application to collect images that the camera is seeing at any time. (False positive)

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific permission that was flagged out by MobSF:
`android.permission.CAMERA`

This is not an issue to the App, because while this permission allows other functions like capturing photos or videos, this permission is used in a way such that it can only be used to scan QR codes. It does not take photos or videos. The app requires this permission to have access to the camera hardware to scan QR codes.

More information
This is a runtime permission that an Android application must request from users to access and utilize the device's camera hardware. It can be misused to record video or take pictures if not handled responsibly.

```
<uses-permission android:name="android.permission.VIBRATE"/>
<!-- Added this for react native vision camera -->
<uses-permission android:name="android.permission.CAMERA" />
```

One common misuse is apps accessing the camera while running in the background. To combat this, Android 9 and later versions introduced privacy changes that prevent apps from accessing the camera when they are backgrounded.

- **Foreground Services Requirement** - When does your app interact with the camera? On Android 9 (API level 28) and later, apps running in the background cannot access the camera. Therefore, you should use the camera either when your app is in the foreground or as part of a foreground service.

https://developer.android.com/about/versions/pie/android-9.0-changes-all